

CLI Usage Examples

Quick Start

```
# Basic usage with default settings (hybrid matcher, 50 repos, deep analysis)
npm start
```

Matcher Selection

Fuzzy Matcher

Uses Topcoder's fuzzy match API to match languages to skills based on string similarity.

```
node dist/cli.js --matcher fuzzy
```

When to use: - Fast execution - Language-based matching - No AI dependencies

Semantic Matcher

Uses AI providers (OpenAI, Gemini, Ollama) for intelligent GenAI-powered matching.

```
# Using OpenAI (cloud, most accurate)
node dist/cli.js --matcher semantic --ai-provider openai --openai-key sk-...

# Using Google Gemini (cloud)
node dist/cli.js --matcher semantic --ai-provider gemini --gemini-key YOUR_KEY

# Using Ollama (local, privacy-focused)
node dist/cli.js --matcher semantic --ai-provider ollama

# Without AI provider (keyword matching)
node dist/cli.js --matcher semantic
```

When to use: - GenAI reasoning for better context understanding - Match based on project descriptions and topics - AI-powered LLM analysis

AI Provider Comparison:

Provider	Accuracy	Speed	Cost	Privacy
OpenAI	★★★★★★	Fast	~\$0.20	Cloud
Gemini	★★★★★	Fast	~\$0.15	Cloud
Ollama	★★★★★	Medium	Free	100% Local
None	★★★★	Fast	Free	N/A

Hybrid Matcher (Recommended)

Combines fuzzy and semantic matching with weighted scoring for best results.

```
node dist/cli.js --matcher hybrid
```

When to use: - Most accurate results - Combines multiple signals - Default and recommended option

AI Provider Configuration

OpenAI Setup

```
# Set API key as environment variable
export OPENAI_API_KEY=sk-...
node dist/cli.js --matcher semantic --ai-provider openai

# Or pass directly
node dist/cli.js --matcher semantic --ai-provider openai --openai-key sk-...
```

Features: - LLM: GPT-4o-mini for reasoning - Embeddings: text-embedding-3-small - Dual-mode: LLM + embedding fallback - Cost: ~\$0.15-\$0.20 per analysis

Google Gemini Setup

```
# Set API key as environment variable
export GEMINI_API_KEY=your_key
node dist/cli.js --matcher semantic --ai-provider gemini

# Or pass directly
node dist/cli.js --matcher semantic --ai-provider gemini --gemini-key your_key
```

Features: - LLM: Gemini Pro for reasoning - Embeddings: embedding-001 - Dual-mode: LLM + embedding fallback - Cost: ~\$0.10-\$0.15 per analysis

Ollama Setup (Local)

```
# 1. Install Ollama (https://ollama.ai)
# 2. Start Ollama server
ollama serve

# 3. Pull model (first time only)
ollama pull llama3.2

# 4. Run analysis
node dist/cli.js --matcher semantic --ai-provider ollama

# Custom model or URL
node dist/cli.js --matcher semantic --ai-provider ollama \
  --ollama-url http://localhost:11434 \
  --ollama-model llama3.2
```

Features: - LLM: llama3.2 (default) or any Ollama model - Embeddings: nomic-embed-text - 100% local inference - no data leaves your machine - Free - no API costs - Privacy-focused

Available Models:

```
# List installed models
```

```
ollama list
```

```
# Pull additional models  
ollama pull mistral  
ollama pull codellama
```

No AI Provider

Use keyword-based matching without AI:

```
node dist/cli.js --matcher semantic  
# or  
node dist/cli.js --matcher fuzzy
```

Performance Tuning

Quick Analysis (Fast)

Skip deep commit/PR analysis for faster results:

```
node dist/cli.js --no-deep-analysis --max-repos 20
```

Deep Analysis (Comprehensive)

Analyze more repos and commits for thorough results:

```
node dist/cli.js --max-repos 100 --max-commits 50
```

Balanced (Default)

Good balance between speed and accuracy:

```
node dist/cli.js --max-repos 50 --max-commits 30
```

Output Options

Display Results

Control how many skills to show:

```
# Show top 20 skills  
node dist/cli.js --max-skills 20  
  
# Show top 5 skills  
node dist/cli.js --max-skills 5
```

Text Output

Human-readable formatted output (default):

```
node dist/cli.js --output text
```

JSON Output

Machine-readable JSON format:

```
node dist/cli.js --output json
```

Save to File

Save results for later analysis:

```
# Save text output
node dist/cli.js --output text --output-file results.txt

# Save JSON output
node dist/cli.js --output json --output-file analysis.json
```

Verbosity

Normal Mode (Default)

Shows progress and results:

```
node dist/cli.js
```

Verbose Mode

Shows detailed logs including API calls and processing steps:

```
node dist/cli.js --verbose
```

Real-World Scenarios

Scenario 1: Quick Profile Check

Fast analysis for a quick overview:

```
node dist/cli.js \
  --matcher fuzzy \
  --max-repos 20 \
  --no-deep-analysis \
  --max-skills 10
```

Expected time: 1-2 minutes

Scenario 2: Comprehensive Analysis

Deep dive for accurate skill recommendations:

```
node dist/cli.js \
  --matcher hybrid \
  --max-repos 100 \
  --max-commits 50 \
  --max-skills 20 \
```

```
--verbose \  
--output json \  
--output-file full-analysis.json
```

Expected time: 5-10 minutes

Scenario 3: AI-Powered Matching

Use OpenAI for semantic understanding:

```
node dist/cli.js \  
  --matcher hybrid \  
  --ai-provider openai \  
  --openai-key $OPENAI_API_KEY \  
  --max-skills 15 \  
  --output json \  
  --output-file ai-results.json
```

Expected time: 3-5 minutes (with API calls)

Scenario 4: Export for Integration

Generate JSON for integration with other systems:

```
node dist/cli.js \  
  --matcher hybrid \  
  --max-repos 50 \  
  --output json \  
  --output-file topcoder-skills.json
```

Then process the JSON:

```
# Pretty print  
cat topcoder-skills.json | jq '.recommendations[] | {skill: .skillName, confidence: .confidence}'  
  
# Filter high-confidence skills  
cat topcoder-skills.json | jq '.recommendations[] | select(.confidence > 80)'
```

Troubleshooting

Rate Limiting

If you hit Github API rate limits:

```
# Reduce repos analyzed  
node dist/cli.js --max-repos 20 --max-commits 10
```

The tool automatically waits when approaching rate limits.

Long Execution Time

For faster results:

```
node dist/cli.js --no-deep-analysis --max-repos 30
```

Missing Skills

For better coverage:

```
# Use hybrid matcher with more results
node dist/cli.js --matcher hybrid --max-skills 30 --verbose
```

Environment Variables

Create a `.env` file:

```
GITHUB_CLIENT_ID=your_github_oauth_app_client_id
GITHUB_CLIENT_SECRET=your_github_oauth_app_client_secret
OPENAI_API_KEY=sk-your_openai_api_key_optional
```

Or pass directly:

```
OPENAI_API_KEY=sk-... node dist/cli.js --ai-provider openai --matcher semantic
```

Help

View all available options:

```
node dist/cli.js --help
```